

Mastère Spécialisé: High Performance Computing & Artificial Intelligence

ADVANCED MASTER'S THESIS

prepared at

Mines Paris - PSL

presented and defended by

Ossara Elysée DJAODOH

September 2025

Learning Meta-Policies for Graph-Based Agent Orchestration using Reinforcement Learning and Graph-of-Thought Planning

Supervisors

Prof. Youssef Mesri	Mines Paris - PSL
Dr. Mathieu Estratat	SmartTrade Technologies

Contents

Contents	iii
1 Introduction	1
2 Contexts and Motivations	3
2.1 References	4
3 Related Works	5
3.1 Introduction	5
3.2 Reinforcement Learning	5
3.2.1 Q-Learning	7
3.2.2 Deep Q-Learning (DQN)	8
3.2.3 REINFORCE	9
3.2.4 Actor-Critic	10
3.2.5 Proximal Policy Optimization (PPO)	11
3.2.6 Reinforcement Learning from Human Feedback (RLHF)	12
3.2.7 Direct Policy Optimization (DPO)	13
3.3 Graph Neural Network	14
3.4 References	15
4 Methodologies	17
4.1 Introduction	17
4.1.1 LLM Agents	18
4.1.2 Environment	18
4.1.3 Orchestrator	19
4.1.4 Critic	19
4.1.5 Data	19
4.1.6 Policy	20
5 Results	23
5.1 REINFORCE with Moving Average Baseline	23
5.2 REINFORCE with Normalize Advantage	23
5.3 Actor-Critic	23
5.4 Discussion	23
6 Conclusion and Perspectives	25

même niveau

Chapter 1

Introduction

Humans excel at recognizing complex, unstructured patterns. However, they often rely on various tools such as books, search engines, or calculators to supplement their knowledge before reaching a conclusion. Similarly, generative AI models can be trained to use external tools to obtain real-time information or suggest actions in the real world. For example, a model might leverage a database query tool to retrieve specific information, such as a customer's purchase history, allowing it to generate personalized purchasing recommendations. Additionally, in response to a user query, a model might trigger API calls to compose an email to a colleague or complete a financial transaction on their behalf. To do this, the model must not only have access to a set of external tools but also be able to develop strategies and execute tasks autonomously. This combination of reasoning, logic, and access to external information, integrated into a generative AI model, corresponds to the concept of an agent, a system that goes beyond the autonomous capabilities of a standard generative AI model and capable of solving specific tasks.

The advent of agents capable of solving specific tasks naturally led to the next idea: combining agents with different capabilities to solve more complex problems. While a team of specialized agents can tackle challenges beyond the scope of any single one, a significant hurdle remains: effectively planning and orchestrating their collaboration to complete a common goal.

Peut être rajouter : How to devant effectively planning ...

Chapter 2

Contexts and Motivations

SmartTrade [1] and Mines Paris – PSL [2] have formed a strategic collaboration to integrate generative artificial intelligence into businesses. This partnership focuses on concrete use cases and includes a strong commitment to production deployment. This collaboration takes the form of a joint AI research laboratory between SmartTrade [1] and Mines Paris – PSL [2]. As part of this initiative, I am contributing as a Machine Learning research intern, participating in the scientific momentum around generative AI.

Puisque après tu parles de la collaboration passée, je rajouterai bien : [...] initiative, I am contributing, this year, as a Machine Learning [...]

For the past three years, this collaboration between SmartTrade [1] and Mines Paris – PSL [2] has been ongoing. Over the years, several iterations have taken place, beginning with a feasibility study on AI integration within SmartTrade [1], followed by the exploration of language models (LLMs) [3] for various tasks. These models led to the creation of Oracle AI before being further enhanced through the integration of RAG (Retrieval-Augmented Generation) [4]. The objective of our current collaboration is to extend the capabilities of existing models. Given the dynamic and rapidly evolving state of the art in this field, we are now moving towards an abstract multi-agent approach. Before delving deeper into this subject, it is essential to define what an agent is. An agent [5] can be described as a system designed to pursue a specific objective by observing its environment and acting based on the tools at its disposal. An agent is autonomous and capable of operating independently, particularly when it is provided with clear goals.

While a single agent is powerful for a specific task, complex real-world problems often require a diverse set of skills and parallel efforts that exceed the capacity of any lone entity. To tackle these larger challenges, it becomes necessary to deploy multiple agents that can work together. This collaborative framework is known as a multi-agent system.

A multi-agent system (MAS) is a framework composed of multiple autonomous agents that operate and interact within a shared environment. In this classic paradigm, each agent possesses its own objectives and decision-making capabilities, and system-level behavior emerges from their local interactions which can range from cooperation to competition. The primary strength of this approach lies in its ability to tackle complex problems in a decentralized manner, leveraging the distributed intelligence of all its members.

However, this very autonomy and decentralization can present significant challenges. Coordinating agents to achieve a specific, optimal outcome can be difficult, as emergent behavior is not always predictable or efficient. For tasks that require a precise sequence of specialized skills, a lack of central oversight can lead to suboptimal strategies or even failure to complete the objective. To address these limitations and gain greater control and reliability in the problem-solving process, we pivot from the concept of emergent collaboration to that of orchestration.

Instead of a traditional multi-agent system where autonomous agents interact freely, our approach

is centered on a more structured and powerful paradigm: Graph-Based Orchestration. In this model, individual agents function as specialized tools or nodes within a dynamic graph. Their collaboration isn't emergent; it is explicitly directed by an intelligent, overarching strategy. The core of our system is a meta-policy learned through Reinforcement Learning (RL). This high-level policy orchestrates the workflow by selecting which agent to execute at each step to solve a complex task. That require sophisticated planning, the system leverages Graph-of-Thought (GoT) Planning. This allows it to build and reason over a flexible graph of potential steps, enabling it to navigate intricate challenges.

Il manque des références à tout ce que tu cites. Il me manque aussi la raison pour laquelle on est parti sur les GoT. Tu dis simplement ce qu'on a utilisé pas pourquoi on les a utilisés.

The primary advantage of this architecture lies in its ability to explicitly model and optimize complex, multi-step problem-solving processes. By structuring agent collaboration as a graph and using a learned meta-policy to direct the flow, we can dynamically pool the skills of multiple agents to achieve a robust and optimized outcome. Our decision to adopt this paradigm is based on its superior capabilities for structured reasoning, flexibility, and scalability. By enabling a meta-policy to learn the optimal way to coordinate agents, this framework is particularly well-suited for addressing highly intricate challenges where a predefined workflow is insufficient. The field of learning meta-policies for agent orchestration is a recent and rapidly evolving frontier. This state-of-the-art report is dedicated to analyzing this new paradigm to identify the best application strategies for SmartTrade [1].

On n'est plus dans celui là :)

2.1 References

- [1] *SmartTrade*.
<https://smart-trade.net/> 3, 4
- [2] *Mines Paris - PSL*.
<https://www.minesparis.psl.eu/> 3
- [3] Humza Naveed and al. *A Comprehensive Overview of Large Language Models*.
<https://arxiv.org/pdf/2307.06435> 3
- [4] Patrick Lewis and al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*.
<https://arxiv.org/pdf/2005.11401> 3
- [5] Julia Wiesinger and al. *Agents*.
https://ppc.land/content/files/2025/01/Newwhitepaper_Agents2.pdf 3

Chapter 3

Related Works

3.1 Introduction

3.2 Reinforcement Learning

Reinforcement learning is about learning how to map situations (states) to actions in order to maximize a numerical reward signal. The learner (agent) is not told which actions to take, but instead must discover which actions yield the most reward by trying them. In the most interesting and challenging cases, actions may affect not only the immediate reward but also the next situation and, through that, all subsequent rewards. These two characteristics trial-and-error search and delayed reward are the two most important distinguishing features of reinforcement learning. [référence ?](#)

We formalize the reinforcement learning problem using ideas from dynamic systems theory. The basic idea is simply to capture the most important aspects of the real problem facing a learning agent that is interacting with its environment over time to achieve a goal. A learning agent must be able to sense the state of its environment to some extent and must be able to take actions that affect that state. The agent must also have a goal or goals relating to the state of the environment. An agent must be able to learn from its own experience. [référence ?](#)

One of the challenges that arises in reinforcement learning is the trade-off between exploration and exploitation. To obtain a lot of rewards, a reinforcement learning agent must prefer actions that it has tried in the past and found to be effective in producing rewards. But to discover such actions, it has to try actions that it has not selected before. The agent has to exploit what it already knows in order to obtain a reward, but it also has to explore in order to make better action selections in the future. The dilemma is that neither exploration nor exploitation can be pursued exclusively without failing at the task. The agent must try a variety of actions and progressively favor those that appear to be best. On a stochastic task, each action must be tried many times to gain a reliable estimate of its expected reward.

To solve the exploration-exploitation dilemma and learn an optimal strategy (a policy), RL algorithms are generally categorized into two main families, along with hybrid methods that combine their strengths.

Value-Based Methods

Value-based methods focus on learning a value function that estimates the expected return (cumulative future reward) of being in a certain state or taking a specific action in a state. The policy is then derived implicitly by always selecting the action with the highest value. The most common value function is the action-value function, $Q(s, a)$. The goal is to find the optimal function, $Q^*(s, a)$, which is described by the Bellman optimality equation:

$$Q^*(s, a) = \mathbb{E}_{s' \sim P} \left[r + \gamma \max_{a'} Q^*(s', a') \mid s, a \right] \quad (3.1)$$

que sont s,a,E,s',P,r,gamma,a' ? :)

Pourquoi pas même dire comment se lit cette fonction ?

- **Q-Learning** is a foundational value-based algorithm that iteratively updates a table of Q-values for each state-action pair using the agent's experiences.
- **Deep Q-Learning (DQN)** modernizes Q-learning by using a deep neural network to approximate the Q-function, $Q(s, a; \theta)$, allowing it to handle high-dimensional state spaces like images from a video game.

Policy-Based Methods

Policy-based methods directly learn a parameterized policy, $\pi_\theta(a|s)$, which defines a probability distribution over actions for a given state. This approach is particularly effective in continuous action spaces. The algorithm updates the policy's parameters θ by performing gradient ascent to maximize the expected total reward, $J(\theta)$. The general form of the policy gradient is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [\nabla_\theta \log \pi_\theta(a|s) G_t] \quad (3.2)$$

where G_t is the total return from time step t .

Que sont la pyramide à l'envers, le E, le Tau et le a ?

There exists different families of Policy-based methods, please find below three examples :

- **REINFORCE** is a classic Monte Carlo policy gradient algorithm that updates the policy based on the total return collected during an entire episode.
- **Reinforcement Learning from Human Feedback (RLHF)** is a modern paradigm, especially for language models, where the reward signal is first learned from human preference data before being used to fine-tune a policy with RL.
- **Direct Policy Optimization (DPO)** is a more recent and stable alternative to RLHF that directly optimizes the policy using preference data, bypassing the need to explicitly train a separate reward model.

Actor-Critic Methods

Actor-Critic methods combine the strengths of both value-based and policy-based approaches. They use two models that learn simultaneously:

1. The Actor is a policy that controls how the agent behaves π_θ .
2. The Critic is a value function that measures how good the chosen actions are $V_w(s)$.

The critic provides feedback to the actor, leading to more stable and efficient learning. The actor's update is often guided by the advantage function, which indicates how much better an action is compared to the average action from that state:

$$A(s, a) = Q_w(s, a) - V_w(s) \quad (3.3)$$

- **Classic Actor-Critic** methods use simple neural networks (like MLPs) for both the actor and the critic to learn in tandem.
- **Proximal Policy Optimization (PPO)** is a state-of-the-art actor-critic algorithm that enhances training stability by constraining the size of the policy updates, preventing drastic changes that could hurt performance.

3.2.1 Q-Learning

Q-Learning is a foundational model-free, off-policy, value-based algorithm that aims to find the optimal action-value function, $Q^*(s, a)$.

The algorithm's off-policy nature is crucial; it learns the optimal policy π^* while behaving according to a different, exploratory policy, most commonly ϵ -greedy. This strategy involves selecting the action with the highest Q-value with probability $1-\epsilon$ and a random action with probability ϵ , allowing for continuous exploration. The value of ϵ is typically annealed (gradually decreased) over time, shifting the agent's focus from exploration to exploitation as it becomes more confident in its Q-value estimates.

Références.

At its heart, Q-learning is a Temporal Difference (TD) learning method. TD learning is a central concept in reinforcement learning that combines ideas from both Monte Carlo methods (learning from experience) and Dynamic Programming (updating estimates based on other estimates). Instead of waiting until the end of an episode to update its value estimates (like Monte Carlo), a TD method updates its predictions at each time step. It does this by comparing its current estimate of a state's value with a new, slightly better estimate that incorporates the immediate reward and the estimated value of the next state. This process of updating an estimate using another learned estimate is known as bootstrapping.

The core update rule is based on the Temporal Difference (TD) error, which is the difference between the new estimate (the TD Target) and the old one. This error signals how to adjust the previous prediction. The update rule is:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \underbrace{\left[r_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t) \right]}_{\text{Temporal Difference (TD) Error}} \quad (3.4)$$

Here, the term $r_{t+1} + \gamma \max_a Q(s_{t+1}, a)$ is the TD Target. The algorithm nudges the current Q-value estimate, $Q(s_t, a_t)$, towards this more informed target.

In practice, Q-learning is implemented with a Q-table, a matrix where rows represent states and columns represent actions. This table is often initialized with zeros or optimistically with high values to encourage exploration. The algorithm's guaranteed convergence to Q^* holds only for problems that can be formulated as Finite Markov Decision Processes (MDPs).

Dernière fois que le note; il manque des références, partout :)

A Finite MDP is the standard mathematical framework for modeling reinforcement learning problems. It's "finite" because the sets of states, actions, and rewards are all finite and countable. Think of a game like chess or a small grid-world maze: there is a limited number of squares (states) you can be in, a limited number of moves (actions) you can make from any square, and a finite set of outcomes (win, lose, draw). This is in contrast to problems with continuous states or actions, like controlling a robotic arm, where the number of possible positions is infinite.

The primary limitation of Q-learning is the curse of dimensionality; the size of the required Q-table grows exponentially with the number of state and action variables, making it completely intractable for problems with large but finite spaces, and impossible for continuous ones.

Algorithm 1 Sarsamax (Q-Learning)

Require: policy π , positive integer num_episodes, small positive fraction α , GLIE $\{\epsilon_i\}$
Ensure: value function Q ($\approx q_\pi$ if num_episodes is large enough)

- 1: Initialize Q arbitrarily (e.g., $Q(s, a) = 0$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$, and $Q(\text{terminal-state}, \cdot) = 0$)
- 2: **for** $i \leftarrow 1$ to num_episodes **do**
- 3: $\epsilon \leftarrow \epsilon_i$
- 4: Observe S_0
- 5: $t \leftarrow 0$
- 6: **repeat**
- 7: Choose action A_t using policy derived from Q (e.g., ϵ -greedy)
- 8: Take action A_t and observe R_{t+1}, S_{t+1}
- 9: $Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha (R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t))$ ▷ Step 4
- 10: $t \leftarrow t + 1$
- 11: **until** S_t is terminal
- 12: **end for**
- 13: **return** Q

3.2.2 Deep Q-Learning (DQN)

Deep Q-Learning (DQN) was a landmark development that successfully scaled Q-Learning to high-dimensional state spaces, allowing agents to learn complex policies directly from raw sensory inputs like pixels. It achieves this by approximating the action-value function, $Q(s, a)$, with a deep neural network, referred to as a Deep Q-Network and parameterized by weights θ , i.e., $Q(s, a; \theta)$.

To gather the experience needed to train this network, the DQN agent must balance exploration and exploitation. It uses an ϵ -greedy strategy to select actions. For a given state s_t , the agent behaves as follows:

- With probability $1 - \epsilon$, it exploits its current knowledge by choosing the action with the highest predicted Q-value:

$$a_t = \arg \max_a Q(s_t, a; \theta)$$

- With probability ϵ , it explores by selecting a completely random action.

The value of ϵ is typically annealed (gradually decreased) over the course of training. It starts high (e.g., $\epsilon = 1.0$) to encourage broad exploration and is lowered to a small constant (e.g., $\epsilon = 0.1$) as the agent's Q-function becomes more reliable, shifting the focus to exploitation.

However, combining neural networks with the correlated data from an agent's experience introduces significant instability. Classical DQN addressed this with two core innovations that have become foundational in the field: **Experience Replay** and a **Target Network**.

- **Experience Replay** involves storing the agent's past transitions $(s_t, a_t, r_{t+1}, s_{t+1})$ in a large memory buffer and sampling random mini-batches from it for training. This breaks the temporal correlations in the data and improves learning efficiency.
- The **Target Network** addresses the “moving target” problem by using a separate, periodically updated copy of the main network, $Q(s, a; \theta^-)$, to provide stable target values for the learning updates.

The DQN algorithm then learns by minimizing the Mean Squared Error between the Q-value predicted by the main network and the stable target value generated by the target network. The loss function is defined as:

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right]. \quad (3.5)$$

Algorithm 2 Deep Q-learning with Experience Replay

```

1: Initialize replay memory  $\mathcal{D}$  to capacity N
2: Initialize action-value function Q with random weights  $\theta$ 
3: for episode = 1, M do
4:   Initialise sequence  $s_1 = \{x_1\}$  and preprocessed sequence  $\phi_1 = \phi(s_1)$ 
5:   for  $t = 1, T$  do
6:     With probability  $\epsilon$  select a random action  $a_t$ 
7:     otherwise select  $a_t = \max_a Q(\phi(s_t), a; \theta)$ 
8:     Execute action  $a_t$  in emulator and observe reward  $r_t$  and image  $x_{t+1}$ 
9:     Set  $s_{t+1} = s_t, a_t, x_{t+1}$  and preprocess  $\phi_{t+1} = \phi(s_{t+1})$ 
10:    Store transition  $(\phi_t, a_t, r_t, \phi_{t+1})$  in  $\mathcal{D}$ 
11:    Sample random minibatch of transitions  $(\phi_j, a_j, r_j, \phi_{j+1})$  from  $\mathcal{D}$ 
12:    Set  $y_j = \begin{cases} r_j & \text{for terminal } \phi_{j+1} \\ r_j + \gamma \max_{a'} Q(\phi_{j+1}, a'; \theta) & \text{for non-terminal } \phi_{j+1} \end{cases}$ 
13:    Perform a gradient descent step on  $(y_j - Q(\phi_j, a_j; \theta))^2$  according to equation 3
14:  end for
15: end for
    
```

3.2.3 REINFORCE

REINFORCE is a Monte Carlo policy gradient algorithm and a foundational method in modern reinforcement learning. As a policy gradient method, its goal is to directly optimize a parameterized policy, $\pi_\theta(a | s)$, by performing gradient ascent on the expected total reward. It is a Monte Carlo method because it learns only from complete trajectories, waiting until the end of an episode to make any updates.

The learning process occurs in two distinct phases: a **rollout phase**, where the agent follows its current policy to generate a full episode of states, actions, and rewards; and an **update phase**, where it uses this trajectory to improve its policy. For every time step in the episode, the total discounted return is calculated by iterating backward from the final step :

$$G_t = \sum_{k=t}^T \gamma^{k-t} r_k \quad (3.6)$$

The policy parameters are then updated using the gradient of the objective function:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) G_t \right] \quad (3.7)$$

The intuition behind this update is powerful :

- The **score function**, $\nabla_\theta \log \pi_\theta(a_t | s_t)$, is a vector that points in the direction in parameter space that will most increase the probability of taking action a_t from state s_t .
- The **return**, G_t , acts as a signed scaling factor. If the return was high (a successful trajectory), the update pushes the policy parameters in the direction of the score function, making those actions more likely in the future. If the return was low or negative, the update pushes the parameters in the opposite direction, making those actions less likely.

The primary challenge with REINFORCE is the extremely high variance of the gradient estimate. Because the return G_t is the sum of all future rewards, a single high or low reward can drastically affect the credit assigned to all preceding actions in the trajectory, regardless of whether they were truly good or bad. This leads to a noisy learning signal and unstable training.

To mitigate this, a state-dependent **baseline**, $b(s_t)$, is subtracted from the return. This does not change the expected value of the gradient but can drastically reduce its variance :

$$\nabla_{\theta} J(\theta) \propto \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)) \quad (3.8)$$

A simple approach to implementing a baseline is to use a moving average of returns. In this method, the algorithm maintains a running average, $\bar{G}$, of all the episodic returns it has seen so far. The update for an action is then scaled by $(G_t - \bar{G})$. While this is very easy to implement, as it doesn't require training a separate network, it is much less informative. It compares a trajectory's outcome to the global average of all past trajectories, ignoring the crucial context of the starting state.

To address this lack of context, the most common and effective choice for a baseline is a learned estimate of the state-value function, $V(s_t)$. By subtracting the expected return from a specific state, the learning signal is centered far more effectively. The update no longer asks, "Was the outcome good?" but rather, "Was the outcome better than expected for this state?" The term $(G_t - V(s_t))$ is an estimate of the **Advantage Function**, $A(s_t, a_t)$, which provides a much more stable signal for learning and is the core concept that leads directly to Actor-Critic methods.

3.2.4 Actor-Critic

Actor-Critic methods represent a powerful hybrid architecture that combines the strengths of policy-based and value-based learning to create a more stable and efficient algorithm. Backed by the Actor-Critic Theorem, this approach was developed as a direct solution to the high-variance problem inherent in pure policy gradient methods like REINFORCE. The architecture can be intuitively understood through an analogy: an Actor performs on a stage, and a Critic provides feedback from the audience. The actor learns to improve its performance based on the critic's judgment, which is more nuanced than simply waiting for the final applause.

The architecture is a synthesis of the two main families of RL algorithms, consisting of two neural networks that are trained concurrently :

- **The Actor (Policy-Based):** The Actor is the policy-based component of the model. Its goal is to learn a parameterized policy, $\pi_{\theta}(a | s)$, that directly maps states to a probability distribution over actions. It controls the agent's behavior and actually makes decisions.
- **The Critic (Value-Based):** The Critic is the value-based component. Its job is to learn a value function, typically the state-value function $V_w(s)$. It evaluates the quality of the states visited by the actor by estimating the expected cumulative reward from that state.

The learning process is a synergistic loop. At each time step, the actor takes an action a_t in state s_t , leading to a reward r_{t+1} and a new state s_{t+1} . The critic then evaluates this transition by calculating the Temporal Difference (TD) error, δ_t :

$$\delta_t = \underbrace{r_{t+1} + \gamma V_w(s_{t+1})}_{\text{TD Target}} - \underbrace{V_w(s_t)}_{\text{Baseline}} \quad (3.9)$$

A positive TD error is a "pleasant surprise," indicating the action led to a better-than-expected outcome, while a negative TD error is a "disappointment."

The effectiveness of this architecture is justified by the idea behind Actor-Critic Theorem. This theorem builds upon the Policy Gradient Theorem, which states that the policy gradient is proportional to the expected value of the score function multiplied by the action-value function $Q^\pi(s, a)$. The Actor-Critic Theorem proves that subtracting a state-dependent baseline $b(s)$ from the action-value function does not change the expected gradient:

$$\nabla_\theta J(\theta) \propto \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) (Q^\pi(s, a) - b(s))] \quad (3.10)$$

The ideal choice for this baseline is the state-value function, $b(s) = V^\pi(s)$, which is precisely what the Critic learns. By subtracting the critic's value estimate from the true action-value, we get the Advantage Function:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (3.11)$$

The advantage function tells us how much better it is to take a specific action a compared to the average action from state s . Using the advantage function as the learning signal dramatically reduces the variance of the gradient estimates compared to using raw returns.

The TD error, δ_t , calculated by the critic is a practical, one-step estimate of this theoretically-sound advantage function. It provides the actor with an immediate, low-variance signal about the quality of its recent action.

Both networks learn from the TD error, each in its respective manner:

- **Critic Update:** The Critic updates its weights w in a value-based fashion, using the TD error to improve its predictions about state values, typically by minimizing a mean squared error loss.
- **Actor Update:** The Actor updates its policy parameters θ using a policy gradient approach. It uses the stable TD error signal from the critic as a practical, low-variance estimate of the advantage function:

$$\theta \leftarrow \theta + \alpha_\theta \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t \quad (3.12)$$

This update pushes the policy to increase the probability of actions that led to pleasant surprises (positive advantage) and decrease the probability of those that led to disappointments (negative advantage).

3.2.5 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a state-of-the-art actor-critic algorithm that has become a default choice in many reinforcement learning applications. It is celebrated for its excellent balance of performance, stability, and ease of implementation. The core design philosophy behind PPO is to address the primary challenge of standard policy gradient methods: their sensitivity to step size. A single large, poorly chosen update can catastrophically damage the policy, leading to a performance collapse. PPO prevents this by ensuring that policy updates are kept small and within a reliable "trust region", which makes the training process significantly more stable.

The core innovation of PPO is its **clipped surrogate objective function**. This objective prevents destructively large policy updates by discouraging the policy from changing too aggressively in a single step. The objective is formulated using the probability ratio between the new and old policies:

$$r_t(\theta) = \frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \quad (3.13)$$

The clipped objective is defined as:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (3.14)$$

Here, $\hat{A}_t$ is the estimated advantage and ϵ is a small hyperparameter (e.g., 0.2). The min function creates a pessimistic bound on the objective, effectively creating a “trust region”. The clipping mechanism works as follows :

- **When advantage is positive** ($\hat{A}_t > 0$): The objective becomes $\min(r_t(\theta)\hat{A}_t, (1 + \epsilon)\hat{A}_t)$. The update is capped if $r_t(\theta)$ grows beyond $1 + \epsilon$, preventing the policy from becoming overconfident based on a single good batch of data.
- **When advantage is negative** ($\hat{A}_t < 0$): The min function takes the maximum of the two terms. The update is bounded if $r_t(\theta)$ shrinks below $1 - \epsilon$, preventing an overly pessimistic update that could destabilize the policy.

By removing the incentive for the policy to move outside this bounded region, PPO ensures more stable and reliable training.

In practice, PPO’s performance is greatly enhanced by using a sophisticated technique for estimating the advantage function called **Generalized Advantage Estimation (GAE)**. GAE balances the trade-off between bias and variance in the advantage estimate:

$$\hat{A}_t^{\text{GAE}(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \quad (3.15)$$

where δ_{t+l} is the TD error at a future time step. The hyperparameter $\lambda \in [0, 1]$ controls the bias-variance trade-off:

- $\lambda = 0$: The GAE estimate reduces to the simple one-step TD error, $\hat{A}_t = \delta_t$. This has low variance but can be highly biased due to reliance on the value function $V(s)$.
- $\lambda = 1$: The GAE estimate becomes the unbiased Monte Carlo return minus the value function. This has low bias but high variance as it depends on long trajectory outcomes.

By choosing an intermediate value (e.g., $\lambda = 0.95$), GAE blends estimates over multiple time steps, providing a more stable and accurate learning signal for the actor. The combination of the clipped surrogate objective and GAE is key to PPO’s robustness and sample efficiency.

3.2.6 Reinforcement Learning from Human Feedback (RLHF)

Reinforcement Learning from Human Feedback (RLHF) is a sophisticated paradigm for aligning powerful generative models with complex, nuanced human values that are difficult to specify in a traditional reward function. It addresses the challenge of teaching an AI abstract goals like “be helpful and harmless” by learning a reward signal directly from human preferences. The process is typically broken down into three distinct stages.

1. Supervised Fine-Tuning (SFT):

The process begins with a general-purpose, pre-trained language model. This base model is first fine-tuned on a small, high-quality dataset of curated prompt-response pairs created by human labelers. This stage adapts the model to the desired output style and domain (e.g., instruction-following or conversational chat). The result is the SFT model, π_{SFT} , which serves as a well-behaved starting point for the subsequent stages.

2. Reward Model (RM) Training:

This is the core of learning from human preferences. To train a reward model (RM), a prompt is fed to the SFT model, which generates several different responses. A human labeler then ranks these responses from best to worst. This ranking data is used to train a separate model, $R_\phi(s, a)$, that predicts a scalar score representing human preference. The RM is typically trained on pairwise comparison data using a loss function based on the Bradley-Terry model, which models the probability that a preferred response y_w is scored higher than a dispreferred response y_l .

3. RL Fine-Tuning with KL Penalty:

In the final stage, the SFT model is treated as a policy, π_θ , and is fine-tuned using a reinforcement learning algorithm almost always Proximal Policy Optimization (PPO). The “environment” is the space of all possible prompts, and the “reward” for a given response is the score provided by the learned reward model. A crucial component is a KL-divergence penalty. The final reward signal is a combination of the RM score and a penalty term that discourages the policy π_θ from deviating too far from the initial SFT policy π_{SFT} :

$$R_{\text{final}}(s, a) = R_\phi(s, a) - \beta \log \frac{\pi_{\text{SFT}}(a | s)}{\pi_\theta(a | s)} \quad (3.16)$$

This penalty is essential to prevent reward hacking, where the policy finds an adversarial output that exploits flaws in the reward model to get a high score (e.g., by being repetitive or overly verbose) while degrading language quality. It also helps maintain the model’s general capabilities and ensures it doesn’t forget the lessons from the SFT stage.

The quality of an RLHF system is highly dependent on the quality and diversity of the human preference data. Biases in the labeler pool or instructions can lead to a biased reward model and, consequently, a biased final policy. Furthermore, the process is susceptible to reward model drift; as the policy is optimized, it may produce outputs that are far outside the distribution of the data the reward model was trained on, making the reward signal unreliable for these novel, high-quality responses. Due to these challenges, the RLHF process is often performed iteratively, with new preference data being collected on the latest policy to train improved reward models over time.

3.2.7 Direct Policy Optimization (DPO)

Direct Policy Optimization (DPO) is a recent and powerful paradigm for preference-based alignment that offers a more direct, stable, and efficient alternative to the complex multi-stage pipeline of Reinforcement Learning from Human Feedback (RLHF). It represents a significant shift in thinking: instead of first training a reward model and then using RL to optimize it, DPO optimizes the policy to directly satisfy the human preference data in a single, stable training stage.

The theoretical foundation of DPO comes from a direct mathematical relationship between the reward function and the optimal policy within the RLHF framework. The derivation shows that the objective of maximizing a reward function while staying close to a reference policy (the goal of RLHF’s final stage) can be achieved without ever explicitly learning the reward function itself. DPO leverages this insight to re-frame the alignment problem from reinforcement learning to a simple classification-style loss. This allows the policy to be moved directly towards behaviors that are consistent with the collected preference data.

The DPO loss function directly compares the log-probabilities that the trained policy, π_θ , and a reference policy, π_{ref} (usually the SFT model), assign to the winning (y_w) and losing (y_l) responses from a human preference dataset:

$$L_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim D} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right] \quad (3.17)$$

The intuition behind this formula is as follows:

- **Log-Probability Ratios** : Terms like $\log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)}$ measure how much the current policy has increased the probability of the preferred response compared to the original SFT model.
- **Maximizing the Difference** : The core of the loss function is to maximize the difference between this ratio for the winning response and the ratio for the losing response.

- **The Objective** : The loss function encourages the policy to increase the relative log-probability of the winning response over the losing one. The log-sigmoid function ($\log \sigma$) smoothly converts this objective into a well-behaved loss that can be optimized with standard gradient descent.

The hyperparameter β acts as a temperature parameter, controlling how strongly the policy is optimized to match the preference data. A higher β leads to a policy that more closely adheres to the preferences, while a lower β results in a policy that stays closer to the reference model.

DPO's main advantages are its simplicity, stability, and efficiency :

- By removing the need to fit a separate reward model, it eliminates a potential source of error and instability and simplifies the engineering pipeline.
- The training process is more stable than the RL-based optimization in RLHF, which can be sensitive to hyperparameters and prone to reward hacking.

However, DPO is still highly dependent on the quality of the initial SFT model and the preference dataset. It is a fine-tuning method that excels at refining an already competent model to better align with human values. It is most effective when the desired behavior is a relatively small deviation from the reference policy's behavior. If a more drastic change in capability is needed, the multi-stage RLHF process may still be more suitable.

Après lecture de la section 3.2, je pense qu'elle est mal présentée, je me suis perdu dans les approches que tu présentes. Elles sont listées au dessus et ensuite approfondies mais pas dans le même ordre et sans lien entre elles, je les aurais groupé pas sous section.

3.3 Graph Neural Network

Graphs are a widely used data structure and a universal language for modeling complex systems. In their most common form, graphs are simply a collection of objects (which are the nodes) and interactions (which are the edges) between pairs of these objects. For example, to encode a city map as a graph, one can use nodes to represent intersections and edges to represent the roads that connect them. The power of the graph formalism lies both in its emphasis on the relationships between points (rather than on the properties of each individual point) and in its generality.

Formally, a graph $G = (V, E)$ is defined by a set of nodes V and a set of edges E between these nodes. We denote an edge from node $u \in V$ to node $v \in V$ by $(u, v) \in E$.

A Graph Neural Network is a general framework for defining deep neural networks on graph data. The main idea is that we want to generate node representations that truly depend on the structure of the graph, as well as any feature information we have. To generate this node representation, we use an encoder. The challenge in developing encoders for graph-structured data is that classic deep learning techniques do not apply. To define a deep neural network on general graphs, we need to define a new type of deep learning architecture.

One way to define a deep neural network on graphs would be to simply use the adjacency matrix as input to a deep neural network. For example, to generate an embedding of an entire graph, we could simply use the adjacency matrix as input to a multilayer perceptron (MLP). The problem with this approach is that it depends on the order of the nodes in the adjacency matrix. In other words, such a model is not permutation invariant, and one of the essential criteria for designing neural networks on graphs is that they must be permutation invariant.

To solve this problem, instead of representing our graph using only the features of each node, the representation of each node will be calculated from its own features and those of its neighbors. Since the set of a node's neighbors does not change when re-labeling or re-ordering the nodes in a matrix, this approach, known as neighborhood aggregation, is naturally permutation invariant. To implement this aggregation, a fundamental mechanism of GNNs called Neural Message Passing is used. This process generally occurs in two key steps:

Non défini avant.

Pourquoi un encodeur ?
Qu'est ce que ça apporte ?
C'est quoi un encodeur ?

Quand tu dis "we"
Tu veux dire que c'est ce qu'on a fait ?
Dans le paragraphe suivant, tu expliques donc ce qu'on a fait.

Mais sans jamais opposer d'autres pistes possibles.
Quelles auraient-elles pu être ?

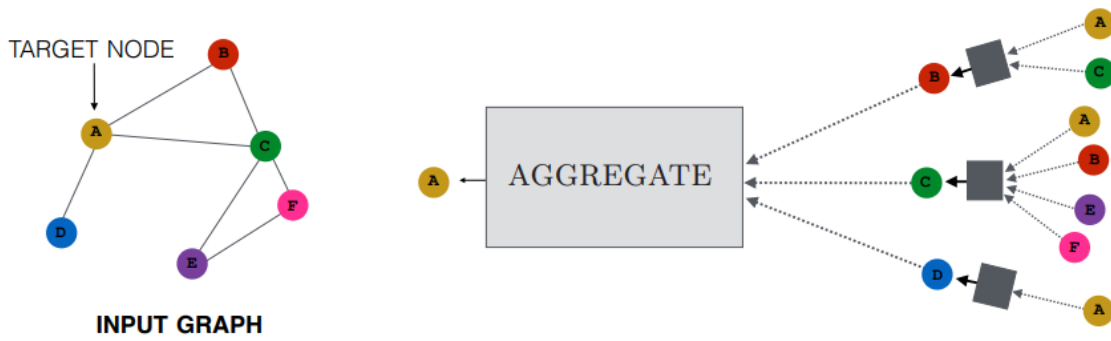


Figure 3.1: Overview of how a single node aggregates messages from its local neighborhood.

Aggregation: Each node collects the feature vectors (the "messages") from its neighboring nodes. To ensure permutation invariance, this aggregation operation must be a symmetric operator, meaning it is insensitive to the order of the neighbors. Functions like sum, mean, or max are commonly used.

Update: The target node's feature vector is then updated by combining its own current vector with the aggregated message from its neighbors. This update is typically performed using a neural network, allowing the model to learn complex transformations.

3.4 References

- [1] Rafael Rafailov and al. *Direct Preference Optimization : Your Language Model is Secretly a Reward Model*. <https://arxiv.org/pdf/2305.18290>
- [2] John Schulman and al. *Proximal Policy Optimization Algorithms*. <https://arxiv.org/pdf/1707.06347>
- [3] John Schulman and al. *HIGH-DIMENSIONAL CONTINUOUS CONTROL USING GENERALIZED ADVANTAGE ESTIMATION*. <https://arxiv.org/pdf/1506.02438>
- [4] Long Ouyang and al. *Training language models to follow instructions with human feedback*. <https://arxiv.org/pdf/2203.02155>

Chapter 4

Methodologies

4.1 Introduction

To overcome the inherent limitations of classic multi-agent systems and to implement a collaboration architecture that is both robust and optimizable, our solution relies on an explicit, graph-based orchestration.

This paradigm allows for learning an orchestration policy (denoted as π) via Reinforcement Learning (RL) in order to optimize the collaboration between the different nodes. The resulting architecture is not only modular and interoperable, enabling the flexible integration of agents (LLMs) and heterogeneous tools, but it also ensures that their combined skills are mobilized in a dynamic and optimal way to solve a given task.

Our approach consists of formulating the orchestration problem as a Markov Decision Process (MDP), which requires rigorously defining its fundamental components:

- **State (s_t)** : The state of our system is represented by the very structure of the collaboration graph at a given time t . This graph captures the available agents, their connections, and their internal states. Since a graph is a complex and non-linear structure, its representation requires a specific approach.
- **Actions (a_t)** : The actions that our orchestration policy can currently undertake are simple operations aimed at modifying the graph. For example: inverting the graph, swapping the positions of two nodes within the graph, or not modifying the graph.
- **Reward (r_t)** : Defining an explicit reward function for a complex collaboration task is difficult. To overcome this obstacle, we use an LLM configured as a reward model. After an action or a sequence of actions, this LLM evaluates or criticizes the relevance of the output resulting from the graph's execution for a given task and returns a scalar reward.

Since the state of our environment is a graph, it cannot be directly processed by standard neural architectures like Multi-Layer Perceptrons (MLPs). We therefore use a Graph Neural Network (GNN) to encode the state s_t . The role of the GNN is to generate an embedding that captures the topological and semantic properties of the collaboration graph. This embedding then serves as the state representation that is provided as input to our RL agent's policy. The execution of an action a_t by the policy modifies the graph, leading to a state transition from s_t to s_{t+1} . The GNN then encodes this new state for the next decision-making step.

Although the state-of-the-art often leans towards complex algorithms like PPO (used in the context of RLHF) or DPO, we have deliberately chosen more fundamental methods such as REINFORCE and Actor-Critic. This choice is motivated by several pragmatic reasons:

1. **Implementation Simplicity** : These algorithms are more straightforward to implement and debug, which accelerates the experimentation cycle.

Ce paragraphe est très bien écrit !

2. **Data Collection Cost** : Methods like DPO or RLHF require human preference datasets, which are extremely expensive and slow to produce. Our approach, relying on a reward model and on-policy algorithms, is much more data-efficient.
3. **Computational Load** : RLHF is a resource-intensive, multi-stage procedure (supervised fine-tuning, reward model training, and then PPO optimization). The REINFORCE and Actor-Critic algorithms have a significantly smaller computational footprint.

4.1.1 LLM Agents

As indicated above, we are looking to learn a Meta-Policy for graph-based agent orchestration using Reinforcement Learning.

To avoid any ambiguity between agents, which can be LLMs or Tools (functions or procedures) and which form the nodes of our collaboration graph, and the agent which is a RL agent capable of perceiving and interacting with our environment, we will now refer to the former as “Agents” and the latter as “policy”. *Moins bien écrit, tu présentes Agents et policy ici, puis tu dis il y a 2 rôles principaux ici*

The components of this system must be clearly defined to avoid ambiguity.

We distinguish between two primary roles:

- **Policy** : This is the “brain” of our system. It is a single Reinforcement Learning agent whose goal is to observe the state of the collaboration graph and select actions to manipulate it such as activating nodes or directing data flow—to solve a given problem.
- **Agents** : These are the “hands” of the system, representing the nodes within the collaboration graph that the Orchestrator manages. Each worker is a specialized component with a single, well defined capability. We categorize them into two types:
 LLM agents : Specialized language models fine-tuned for a specific skill (e.g., a CoderAgent for Python, a ReviewerAgent for text analysis).
 Tools : Deterministic, non-LLM functions or procedures (e.g., a Calculator, a DatabaseSearchAPI, or a CodeExecutor).

4.1.2 Environment

In reinforcement learning, the environment represents the world in which an agent operates. It is a core component of the system, responsible for presenting the agent with a state, receiving an action, and in response, returning a numerical reward and the next state.

In our framework, the environment is specifically designed to manage and evaluate the collaboration between a group of Agents. The components of our environment are defined as follows:

- **State** (s_t) : The state of our environment is the collaboration graph that orchestrates the Agents. This graph defines the workflow, data flow, and interactions between the agents for a given task. The initial state of the graph, s_0 , is generated by an Orchestrator LLM that creates an initial plan.
- **Action** (a_t) : An action, taken by our RL agent, corresponds to modifying the collaboration graph. This could involve adding or removing an agent (a node), changing the flow of information (an edge), or altering an agent’s instructions. *La policy pourrait faire ça ? C’est son rôle ?*
- **Transition Reward** ($s_t + 1, r_t + 1$) : After the RL agent takes an action, the environment transitions to a new state, $s_t + 1$, which is the modified graph. The environment then executes this new graph, causing the Agents to collaborate and produce a final output. This output is evaluated by a Critic LLM, which assesses its quality, coherence, and relevance. The Critic’s evaluation is then converted into a numerical reward, $r_t + 1$, which is returned to the RL agent to signal the effectiveness of its last action.

Qui est il ?

4.1.3 Orchestrator

The Orchestrator is a specialized LLM responsible for creating the initial plan for solving a given task. It takes two inputs: the user's task description and a predefined roster of available Agents with known capabilities.

Based on this information, the Orchestrator analyzes the task and performs two key steps:

- **Agent Selection** : First, it selects the most suitable Agents for the job. These agents become the nodes of the collaboration graph.
- **Workflow Design** : Second, it designs the workflow and data flow between these agents, defining how they will collaborate to complete the task. These connections form the edges of the graph.

The final output is a Directed Acyclic Graph (DAG). This graph structure is intentionally used because, by definition, it cannot contain cycles. This ensures a clear start and end to the process, preventing endless loops and guaranteeing that the task will terminate.

4.1.4 Critic

A critical component of any RL system is the reward signal, which guides the learning agent by evaluating the quality of its actions. However, for the complex, abstract tasks our system addresses, there is no natural environment to provide this signal. To solve this, we introduce the Critic, a specialized LLM that functions as a simulated reward provider.

The Critic's process is as follows:

- After the agent collaboration graph is executed and produces a final output, this output is passed to the Critic.
- The Critic evaluates the output's quality, coherence, and relevance against the initial task description.
- Based on this evaluation, the Critic generates a structured critique which is then translated into a numerical reward signal (e.g., a score from 0 to 10).

This reward is returned to the RL agent, quantitatively informing it about the success of its recent actions and guiding it toward behaviors that maximize long-term positive outcomes.

4.1.5 Data

The dataset used to train our reinforcement learning (RL) model was generated with the Gemini language model.

This data was specifically structured to cover a wide range of task complexities, determined by the number and combination of specialized agents required for a solution. To achieve this, we generated distinct categories of tasks, where each category necessitates a unique subset of our available agents.

This ranges from simple tasks solvable by a single agent to moderately complex ones requiring the collaboration of two agents, and so on, up to highly intricate problems that demand the co-operation of all n agents. This method ensures our RL model is trained on a diverse curriculum, learning to orchestrate everything from simple actions to complex, multi-agent workflows.

4.1.6 Policy

In our framework, we distinguish between the LLM Agents, which form the nodes of our collaboration graph, and the Policy, which is the intelligent component that learns how to orchestrate them. The Policy observes the state of the environment (the graph), takes an action to modify it, and receives a reward based on the outcome.

To train this orchestrating Policy, we explored a progression of three reinforcement learning algorithms, each building upon the last.

Regardless of the training algorithm, the input to our Policy is processed by a Graph Neural Network (GNN). The state of our environment is the collaboration graph itself. The GNN's role is to create a fixed size embedding a numerical representation of this entire graph structure. This embedding captures the complex relationships between the LLM Agents and serves as the input state representation, s_t , for our learning algorithms. The Policy, in turn, outputs a probability distribution over the set of possible actions (modifications to the graph).

REINFORCE with a Moving Average Baseline

Our initial approach was to use the REINFORCE algorithm with a simple baseline to reduce variance. In this setup, the Policy is a Multi-Layer Perceptron (MLP) that takes the GNN state embedding and outputs action probabilities. The baseline is a moving average of returns ($\bar{G}$), which tracks the average of all total returns seen during training.

After each full episode, the total discounted return, G_t , is calculated for each step. The policy is then updated using the advantage estimate, $(G_t - \bar{G})$.

Limitation: This method still suffers from high variance due to its Monte Carlo nature. Furthermore, the baseline is not state-dependent; all actions are judged against the global average outcome, not against what is expected for their specific states. This can lead to a weak learning signal and poor credit assignment.

REINFORCE with Normalized Advantage

To address the stability issues of our first approach, we implemented an improvement: REINFORCE with Normalized Advantage.

This method builds directly on the previous one, but with a crucial extra step. Instead of using the raw advantage estimate, $(G_t - \bar{G})$, we collect a batch of episodes and then normalize all the calculated advantage values from that batch to have a mean of 0 and a standard deviation of 1.

Benefit and Limitation: This normalization prevents a few episodes with unusually high or low rewards from dominating the gradient update, making the training process more stable. However, this is still a Monte Carlo method. It must wait for episodes to complete, making it sample-inefficient, and it still struggles with assigning credit for actions accurately on a step-by-step basis.

Actor-Critic Architecture

To overcome the limitations of Monte Carlo updates and improve both sample efficiency and credit assignment, we adopted an Actor-Critic architecture. This approach allows the agent to learn in an online, step-by-step manner.

Shared Input Both the Actor and Critic networks receive the same GNN embedding of the current graph state.

The Actor (Policy-Based) The Actor is an MLP that functions as our policy-based component. Its goal is to learn the optimal policy,

$$\pi_{\theta}(a | s),$$

by directly controlling the agent's behavior and deciding which action to take.

The Critic (Value-Based) The Critic is a separate MLP that acts as our value-based component. Its job is not to choose actions, but to evaluate the actions chosen by the Actor by learning the state-value function,

$$V_w(s).$$

Instead of waiting for a full episode, the Critic calculates a Temporal Difference (TD) error at each step. This error provides an immediate, low-variance estimate of the advantage of the action just taken. The Actor then uses this instantaneous feedback to update its policy. This synergistic process allows for much more efficient and targeted learning, as good or bad actions are reinforced immediately.

Chapter 5

Results

5.1 REINFORCE with Moving Average Baseline

5.2 REINFORCE with Normalize Advantage

5.3 Actor-Critic

5.4 Discussion

Chapter 6

Conclusion and Perspectives